date changes). In these cases, we would have to be careful in reusing values we have already computed, because they can go *stale*, meaning that, under some conditions, the value once computed and stored in our cache could not be the most up to date, or even just relevant, anymore.

How we deal with stale cache depends heavily on the context. In some cases, even stale values can provide an acceptable approximation, when a certain margin of error is acceptable. For instance, if we do compute aggregates on daily sales, and show them in a live chart, we might be OK about having data synched every minute or every 10 minutes, and so avoid recomputing all the aggregates (possibly grouped by product or store) every time the chart is displayed in our reporting tool (which could be used by tens of employees at the same time).

Probably the best example explaining why this “controlled staleness” can be fine is provided by web caches: HTTP standards allow the server (the provider of the content) to add headers[13] to resources (web pages, but also images, files etc…) to let the client (usually the browser) know when it is allowed to read a certain resource from cache, and also to let intermediate cache nodes know if[14] and for how long the response provided by the server to an HTTP (GET, usually) call can be cached.

Vice versa, if we have a time-critical, safety-critical application, like a monitoring application for a nuclear power plant (for some reason, this example works well in communicating a sense of criticality…), then we can’t afford